

AI-powered bioinformatic analysis using R, #9

Statistical Analysis & Machine Learning Basics

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-07-18

Table of contents I

- 1 Descriptive Statistics
- 2 Hypothesis Testing I: Comparing Two Groups
- 3 Hypothesis Testing II: Comparing Multiple Groups
- 4 Multiple Testing Correction
- 5 Correlation Analysis
- 6 Clustering
- 7 Dimensionality Reduction
- 8 Machine Learning: Classification
- 9 Cross-Validation & Model Evaluation
- 10 Classification: Types & Evaluation
- 11 Traditional Machine Learning Algorithms
- 12 Deep Learning: CNN-based Modern ML



Table of contents II

13 Summary

Section 1

Descriptive Statistics

Summary Statistics

Summarizing data is the first step of any bioinformatics analysis. Use `summary()` and `dplyr::summarise()`:

```
library(tidyverse)

# Built-in summary
summary(mtcars$mpg)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
10.40	15.43	19.20	20.09	22.80	33.90

Summary Statistics, cont.

```
# dplyr summaries
mtcars %>%
  summarise(
    n      = n(),
    mean   = mean(mpg),
    sd     = sd(mpg),
    median = median(mpg),
    IQR    = IQR(mpg),
    min    = min(mpg),
    max    = max(mpg)
  )
```

	n	mean	sd	median	IQR	min	max
1	32	20.09062	6.026948	19.2	7.375	10.4	33.9

Grouped Summary Statistics

In bioinformatics, we always compare **groups** (e.g., case vs. control, different tissues):

```
# mtcars: compare mpg by cylinder groups
mtcars %>%
  group_by(cyl) %>%
  summarise(
    n      = n(),
    mean   = mean(mpg),
    sd     = sd(mpg),
    .groups = "drop"
  )
```

```
# A tibble: 3 x 4
   cyl     n mean   sd
<dbl> <int> <dbl> <dbl>
1     4    11 26.7  4.51
2     6     7 19.7  1.45
3     8    14 15.1  2.56
```

The tableone Package

For publication-ready Table 1 (baseline characteristics table):

```
# install.packages("tableone")
library(tableone)

vars <- c("mpg", "hp", "wt")
CreateTableOne(vars = vars, strata = "cyl", data = mtcars)
```

	Stratified by cyl				p	test
	4	6	7	8		
n	11			14		
mpg (mean (SD))	26.66 (4.51)	19.74 (1.45)		15.10 (2.56)	<0.001	
hp (mean (SD))	82.64 (20.93)	122.29 (24.26)		209.21 (50.98)	<0.001	
wt (mean (SD))	2.29 (0.57)	3.12 (0.36)		4.00 (0.76)	<0.001	

Section 2

Hypothesis Testing I: Comparing Two Groups

The Logic of Hypothesis Testing

- ❶ **Null hypothesis** (H_0): no difference between groups
- ❷ **Test statistic**: quantify the observed difference
- ❸ **P-value**: probability of observing such a difference (or more extreme) **if H_0 is true**
- ❹ If $p < \alpha$ (usually 0.05), reject $H_0 \rightarrow$ statistically significant

t-test: Compare Means of Two Groups

Assumptions: data are approximately normal, independent samples.

```
# Example: compare mpg between automatic (0) and manual (1) transmissions
auto_mpg  <- mtcars %>% filter(am == 0) %>% pull(mpg)
manual_mpg <- mtcars %>% filter(am == 1) %>% pull(mpg)

t.test(auto_mpg, manual_mpg)
```

Welch Two Sample t-test

```
data:  auto_mpg and manual_mpg
t = -3.7671, df = 18.332, p-value = 0.001374
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -11.280194 -3.209684
sample estimates:
mean of x mean of y
 17.14737  24.39231
```

Paired t-test

Used when samples are paired (e.g., before/after treatment on same subjects):

```
# Simulate paired data
set.seed(42)
before <- rnorm(10, mean = 100, sd = 10)
after  <- before + rnorm(10, mean = -5, sd = 3) # slight decrease

t.test(before, after, paired = TRUE)
```

Paired t-test

```
data:  before and after
t = 3.5493, df = 9, p-value = 0.006223
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 1.991021 8.989720
sample estimates:
mean difference
 5.49037
```

Wilcoxon Rank-Sum Test (Mann-Whitney U)

Non-parametric alternative to t-test — does **not** assume normality. Tests whether two samples come from the same distribution.

```
# Compare mpg by transmission type  
wilcox.test(mpg ~ am, data = mtcars)
```

Wilcoxon rank sum test with continuity correction

```
data:  mpg by am  
W = 42, p-value = 0.001871  
alternative hypothesis: true location shift is not equal to 0
```

Section 3

Hypothesis Testing II: Comparing Multiple Groups

One-way ANOVA

Tests whether means of **three or more** groups are equal:

- H_0 : all group means are equal
- H_1 : at least one group mean is different

```
# Compare mpg across cylinder groups (4, 6, 8)
aov_result <- aov(mpg ~ factor(cyl), data = mtcars)
summary(aov_result)
```

```
          Df Sum Sq Mean Sq F value    Pr(>F)
factor(cyl)  2  824.8   412.4    39.7 4.98e-09 ***
Residuals   29  301.3    10.4
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Post-hoc Test: Tukey HSD

If ANOVA is significant, which groups differ?

```
TukeyHSD(aov_result)
```

```
Tukey multiple comparisons of means
 95% family-wise confidence level
```

```
Fit: aov(formula = mpg ~ factor(cyl), data = mtcars)
```

```
$`factor(cyl)`
```

	diff	lwr	upr	p adj
6-4	-6.920779	-10.769350	-3.0722086	0.0003424
8-4	-11.563636	-14.770779	-8.3564942	0.0000000
8-6	-4.642857	-8.327583	-0.9581313	0.0112287

Kruskal-Wallis Test

Non-parametric alternative to one-way ANOVA (does not assume normality):

```
kruskal.test(mpg ~ factor(cyl), data = mtcars)
```

Kruskal-Wallis rank sum test

data: mpg by factor(cyl)

Kruskal-Wallis chi-squared = 25.746, df = 2, p-value = 2.566e-06

Chi-squared Test (χ^2)

Tests association between **categorical** variables:

```
# Contingency table: gear vs. cyl
(tab <- table(mtcars$cyl, mtcars$gear))
```

```
      3  4  5
4    1  8  2
6    2  4  1
8   12  0  2
```

```
chisq.test(tab)
```

Pearson's Chi-squared test

```
data:  tab
X-squared = 18.036, df = 4, p-value = 0.001214
```

Section 4

Multiple Testing Correction

Why Multiple Testing Matters

In bioinformatics, we often test **thousands** of hypotheses simultaneously (e.g., 20,000 genes). If we use $p < 0.05$ for each, we expect **1,000 false positives** by chance alone!

Solution: adjust p-values to control false positives.

Bonferroni Correction

Very conservative — divide α by number of tests:

$$p_{\text{adjusted}} = \min(p \times n, 1)$$

```
# Simulate 1000 p-values (most from null, some from alternative)
set.seed(123)
pvals <- c(runif(950), rbeta(50, 0.5, 5)) # 50 true signals mixed in

p_bonf <- p.adjust(pvals, method = "bonferroni")

# How many significant before/after?
sum(pvals < 0.05)      # before correction
```

```
[1] 71
```

```
sum(p_bonf < 0.05)      # after Bonferroni
```

```
[1] 0
```

Benjamini-Hochberg (FDR) Correction

Controls the **False Discovery Rate** — the expected proportion of false positives among all discoveries. **Default in most bioinformatics tools** (DESeq2, edgeR, limma).

```
p_bh <- p.adjust(pvals, method = "BH")
```

```
sum(p_bh < 0.05)      # after BH correction
```

```
[1] 0
```

```
# Compare
```

```
data.frame(
  Method = c("Unadjusted", "Bonferroni", "BH (FDR)"),
  Significant = c(sum(pvals < 0.05), sum(p_bonf < 0.05), sum(p_bh < 0.05))
)
```

	Method	Significant
1	Unadjusted	71
2	Bonferroni	0
3	BH (FDR)	0

Visual Comparison of Adjustment Methods

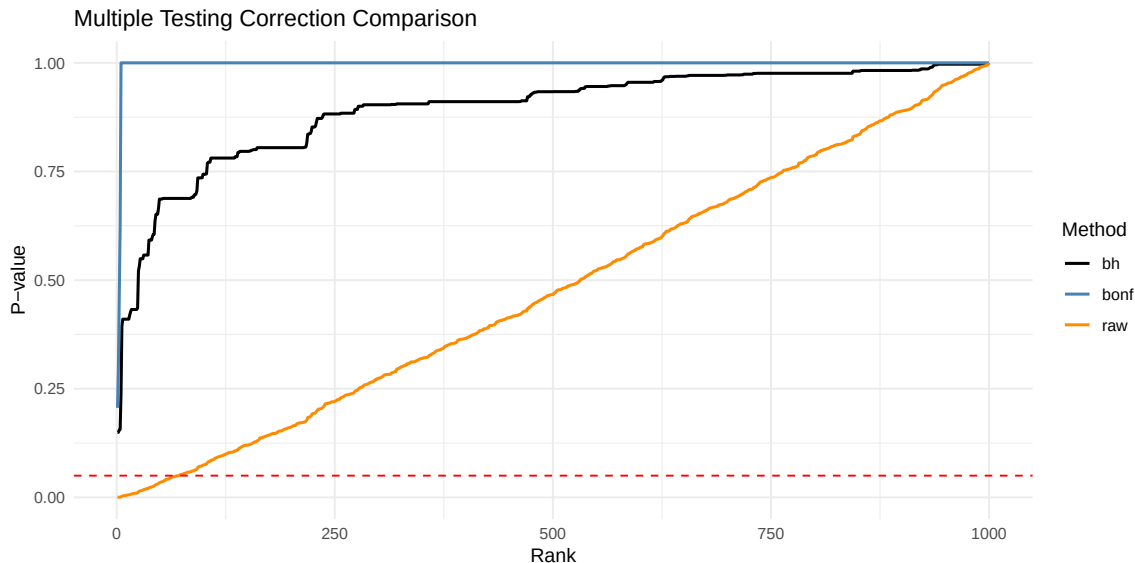
```
library(patchwork)

# Sort p-values and plot
p_df <- tibble(
  rank = 1:1000,
  raw = sort(pvals),
  bonf = sort(p_bonf),
  bh = sort(p_bh)
) %>%
  pivot_longer(cols = c(raw, bonf, bh),
               names_to = "method", values_to = "pvalue")

plot <- ggplot(p_df, aes(x = rank, y = pvalue, color = method)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0.05, linetype = "dashed", color = "red") +
  labs(x = "Rank", y = "P-value",
       title = "Multiple Testing Correction Comparison",
       color = "Method") +
  theme_minimal()
```

Visual Comparison of Adjustment Methods, cont.

```
plot + scale_color_manual(values = c("black", "steelblue", "darkorange"))
```



Section 5

Correlation Analysis

Pearson Correlation

Measures **linear** relationship between two continuous variables:

- $r = 1$: perfect positive correlation
- $r = -1$: perfect negative correlation
- $r = 0$: no linear correlation

```
# Correlation between mpg and hp (horsepower)
cor.test(mtcars$mpg, mtcars$hp, method = "pearson")
```

Pearson's product-moment correlation

```
data:  mtcars$mpg and mtcars$hp
t = -6.7424, df = 30, p-value = 1.788e-07
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.8852686 -0.5860994
sample estimates:
      cor
-0.7761684
```

Spearman Correlation

Rank-based — does not assume linearity, robust to outliers:

```
cor.test(mtcars$mpg, mtcars$hp, method = "spearman")
```

Spearman's rank correlation rho

data: mtcars\$mpg and mtcars\$hp

S = 10337, p-value = 5.086e-12

alternative hypothesis: true rho is not equal to 0

sample estimates:

rho

-0.8946646

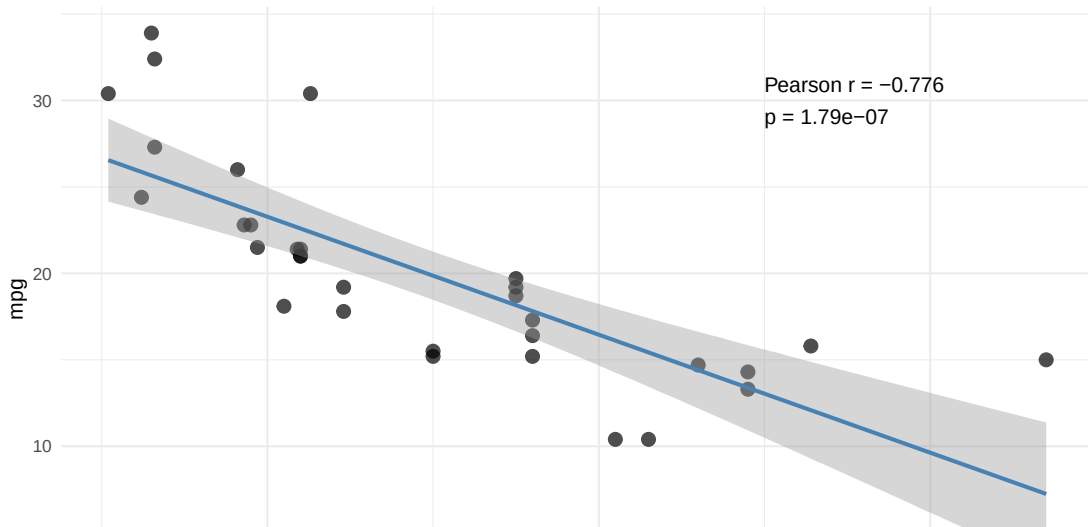
Visualizing Correlations

```
# Scatter plot with correlation
plot <-
ggplot(mtcars, aes(x = hp, y = mpg)) +
  geom_point(size = 3, alpha = 0.7) +
  geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
  annotate("text", x = 250, y = 30,
    label = paste0("Pearson r = ",
      round(cor(mtcars$mpg, mtcars$hp), 3),
      "\np = ",
      format.pval(cor.test(mtcars$mpg, mtcars$hp)$p.value,
        digits = 3)),
    hjust = 0, size = 4) +
  labs(title = "mpg vs. hp: Strong Negative Correlation") +
  theme_minimal()
```

Visualizing Correlations, cont.

```
plot + theme(legend.position = "none")
```

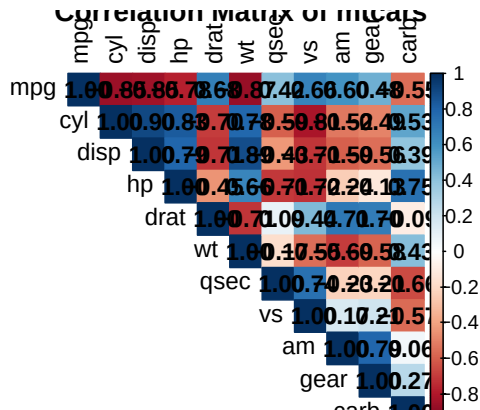
mpg vs. hp: Strong Negative Correlation



Correlation Matrix

For many variables, use a correlation heatmap:

```
library(corrplot)
cor_matrix <- cor(mtcars)
corrplot(cor_matrix, method = "color", type = "upper",
         addCoef.col = "black", tl.col = "black",
         title = "Correlation Matrix of mtcars")
```



Section 6

Clustering

Why Clustering in Bioinformatics?

- Group genes with similar expression patterns
- Identify patient subtypes in clinical data
- Discover microbial community types
- Find co-expressed gene modules

k-means Clustering

Partitions data into **k** pre-specified clusters by minimizing within-cluster variance:

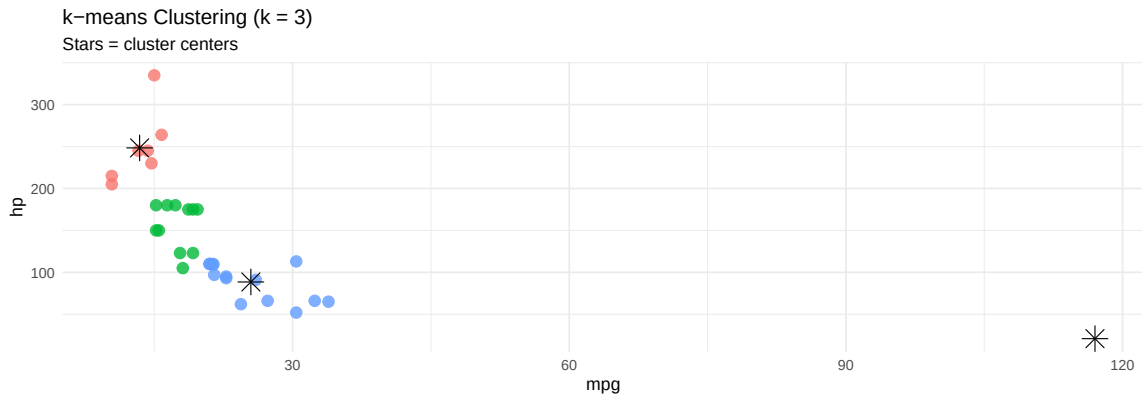
```
set.seed(123)
# Cluster mtcars into 3 groups based on mpg and hp
km <- kmeans(scale(mtcars[, c("mpg", "hp")]), centers = 3)

# Add cluster assignment
mtcars_clust <- mtcars %>%
  mutate(cluster = factor(km$cluster))

plot <-
ggplot(mtcars_clust, aes(x = mpg, y = hp, color = cluster)) +
  geom_point(size = 3, alpha = 0.8) +
  geom_point(data = as_tibble(km$centers *
    attr(scale(mtcars[, c("mpg", "hp")]), "scaled:scale") +
    attr(scale(mtcars[, c("mpg", "hp")]), "scaled:center")),
    aes(x = mpg, y = hp), color = "black", size = 5, shape = 8) +
  labs(title = "k-means Clustering (k = 3)",
    subtitle = "Stars = cluster centers") +
  theme_minimal()
```

k-means Clustering, cont.

```
plot + theme(legend.position = "none")
```



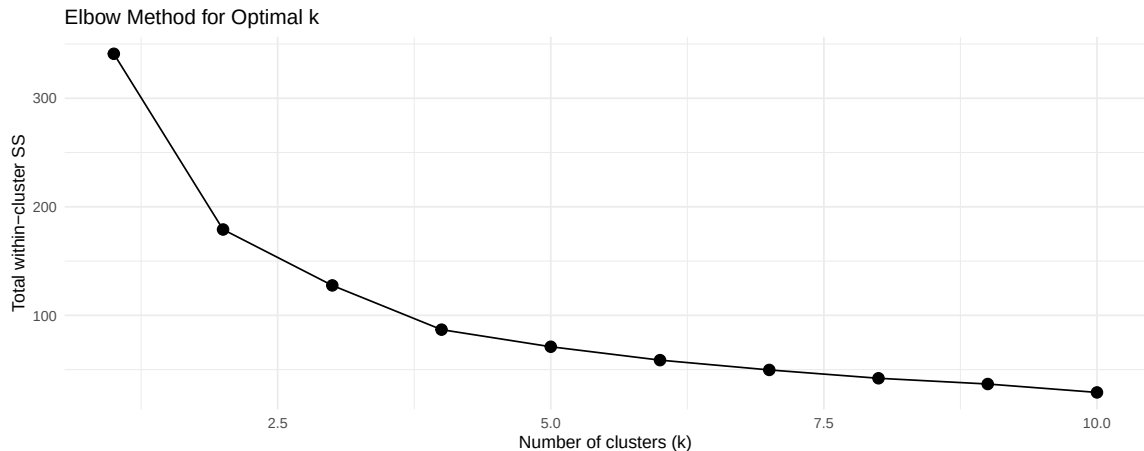
Choosing k: Elbow Method

Plot within-cluster sum of squares (WSS) vs. k — look for the “elbow”:

```
wss <- sapply(1:10, function(k) {  
  kmeans(scale(mtcars), centers = k, nstart = 10)$tot.withinss  
})  
  
plot <-  
ggplot(tibble(k = 1:10, wss = wss), aes(x = k, y = wss)) +  
  geom_line() + geom_point(size = 3) +  
  labs(title = "Elbow Method for Optimal k",  
        x = "Number of clusters (k)", y = "Total within-cluster SS") +  
  theme_minimal()
```

Choosing k: Elbow Method, cont.

```
plot + theme(legend.position = "none")
```

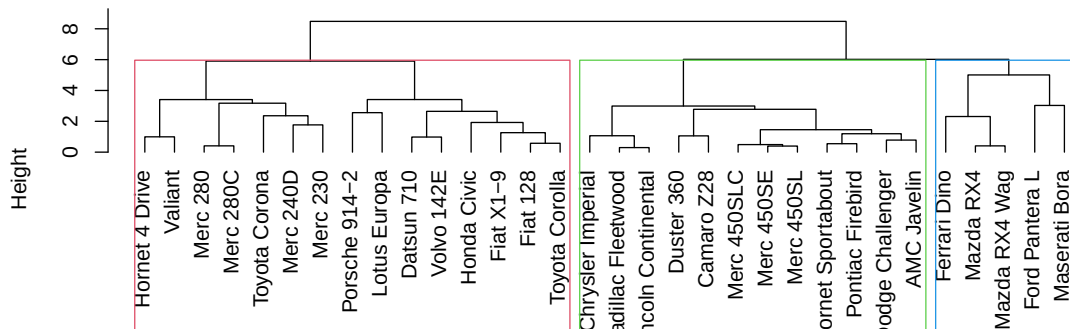


Hierarchical Clustering

Builds a **dendrogram** — does not require pre-specifying k :

```
hc <- hclust(dist(scale(mtcars)), method = "complete") # Hierarchical clustering on mtcars
plot(hc, main = "Hierarchical Clustering of mtcars",
     xlab = "", sub = "", hang = -1) # Plot dendrogram
rect.hclust(hc, k = 3, border = 2:4) # Add rectangle for 3 clusters
```

Hierarchical Clustering of mtcars

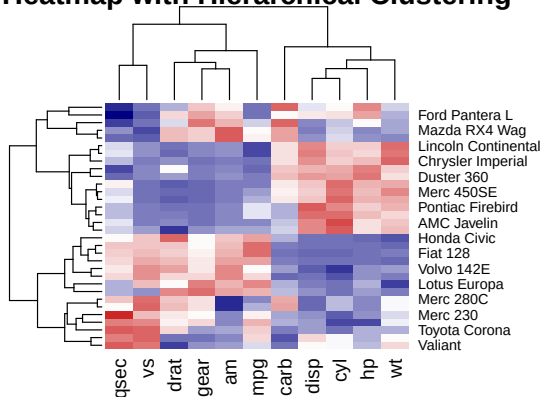


Heatmap + Dendrogram

Combining clustering with expression visualization — a bioinformatics staple:

```
heatmap(scale(mtcars), # Scale and plot heatmap
  main = "Heatmap with Hierarchical Clustering",
  col = colorRampPalette(c("navy", "white", "firebrick3"))(50),
  margins = c(8, 5))
```

Heatmap with Hierarchical Clustering



Section 7

Dimensionality Reduction

Why Reduce Dimensions?

- Visualize high-dimensional data in 2D/3D
- Remove noise and redundancy
- Discover hidden structure
- Essential for: gene expression (20,000+ genes $\rightarrow$ 2D), single-cell data, microbiome

PCA: Principal Component Analysis

Finds directions (principal components) that capture **maximum variance**:

```
# PCA on mtcars
pca <- prcomp(mtcars, scale. = TRUE)

# Variance explained by each PC
summary(pca)$importance[, 1:4]
```

	PC1	PC2	PC3	PC4
Standard deviation	2.570681	1.628026	0.7919579	0.5192277
Proportion of Variance	0.600760	0.240950	0.0570200	0.0245100
Cumulative Proportion	0.600760	0.841720	0.8987300	0.9232400

PCA Visualization: Scree Plot

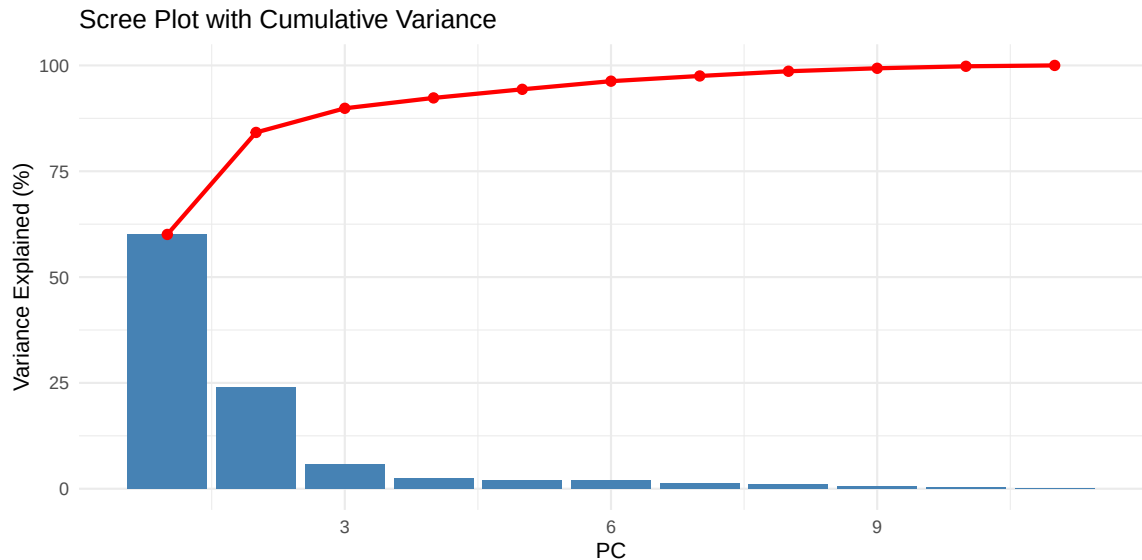
How much variance does each PC explain?

```
# Variance explained
var_exp <- pca$sdev^2 / sum(pca$sdev^2) * 100

plot <-
ggplot(tibble(PC = 1:length(var_exp), Variance = var_exp),
  aes(x = PC, y = Variance)) +
  geom_col(fill = "steelblue") +
  geom_line(aes(y = cumsum(Variance)), color = "red", linewidth = 1) +
  geom_point(aes(y = cumsum(Variance)), color = "red", size = 2) +
  labs(title = "Scree Plot with Cumulative Variance",
    y = "Variance Explained (%)") +
  theme_minimal()
```

PCA Visualization: Scree Plot, cont.

```
plot + theme(legend.position = "none")
```



PCA Visualization: Score Plot

Project samples onto PC1 and PC2:

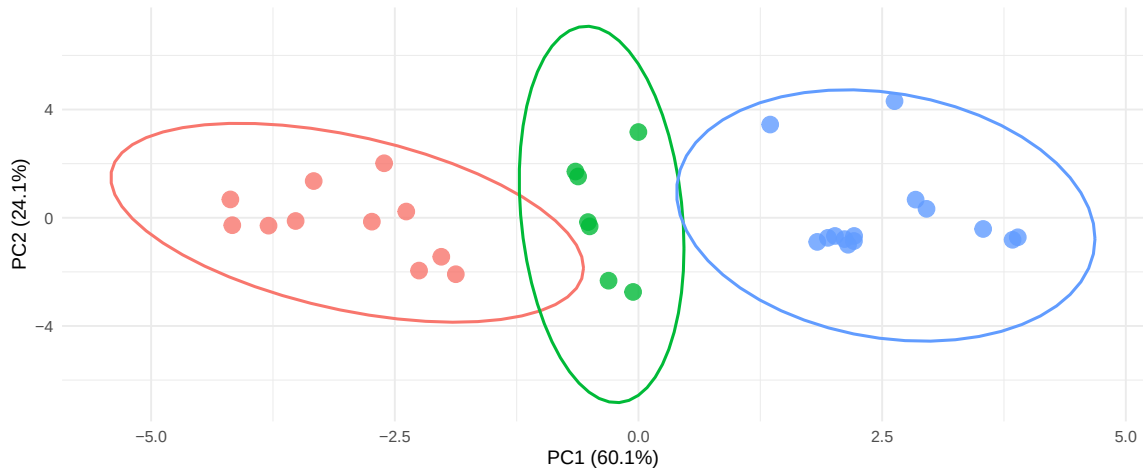
```
# Extract PC scores
pca_scores <- as_tibble(pca$x)
pca_scores$cyl <- factor(mtcars$cyl)

plot <-
ggplot(pca_scores, aes(x = PC1, y = PC2, color = cyl)) +
  geom_point(size = 4, alpha = 0.8) +
  stat_ellipse(type = "norm", linewidth = 0.8) +
  labs(title = "PCA Score Plot: PC1 vs PC2",
       x = paste0("PC1 (", round(var_exp[1], 1), "%)"),
       y = paste0("PC2 (", round(var_exp[2], 1), "%)")) +
  theme_minimal()
```

PCA Visualization: Score Plot, cont.

```
plot + theme(legend.position = "none")
```

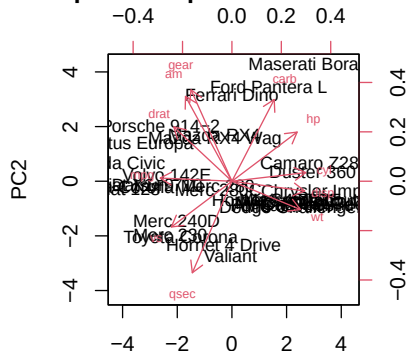
PCA Score Plot: PC1 vs PC2



PCA: Loadings / Biplot

Which variables contribute most to each PC?

```
# Biplot
biplot(pca, scale = 0, cex = c(0.8, 0.6),
       main = "PCA Biplot: Samples + Variable Loadings")
```



t-SNE

t-Distributed Stochastic Neighbor Embedding — preserves local structure, excellent for visualization:

```
# install.packages("Rtsne")
library(Rtsne)

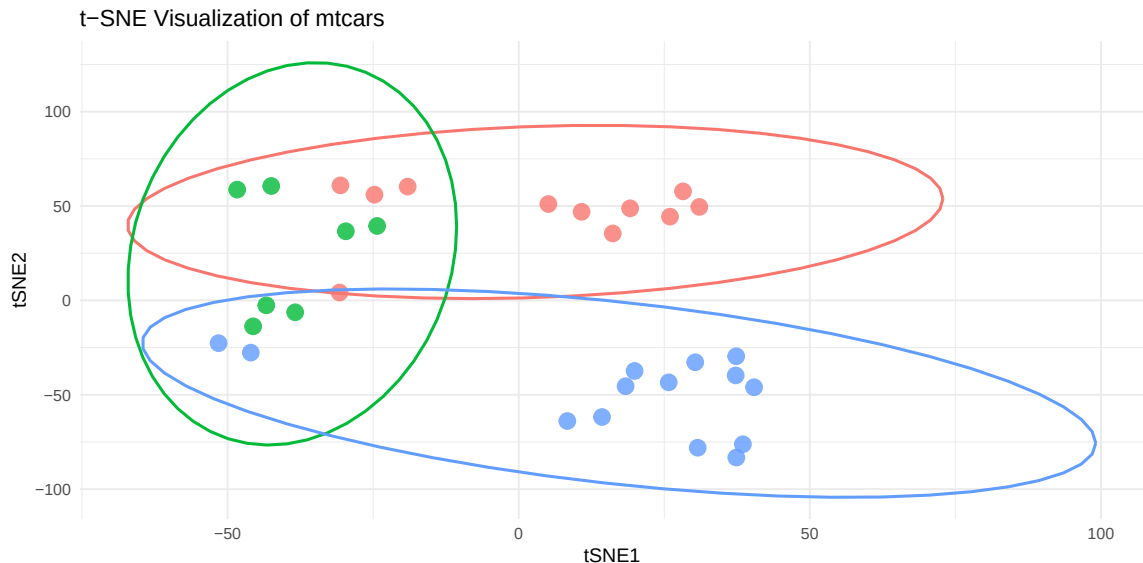
set.seed(42)
tsne_out <- Rtsne(scale(mtcars), perplexity = 5)

tsne_df <- tibble(
  tSNE1 = tsne_out$Y[, 1],
  tSNE2 = tsne_out$Y[, 2],
  cyl   = factor(mtcars$cyl)
)

plot <- ggplot(tsne_df, aes(x = tSNE1, y = tSNE2, color = cyl)) +
  geom_point(size = 4, alpha = 0.8) +
  stat_ellipse(type = "norm", linewidth = 0.8) +
  labs(title = "t-SNE Visualization of mtcars") +
  theme_minimal()
```

t-SNE, cont.

```
plot + theme(legend.position = "none")
```



UMAP

Uniform Manifold Approximation and Projection — faster than t-SNE, preserves both local and global structure:

```
# install.packages("umap")
library(umap)

set.seed(42)
umap_out <- umap(scale(mtcars))

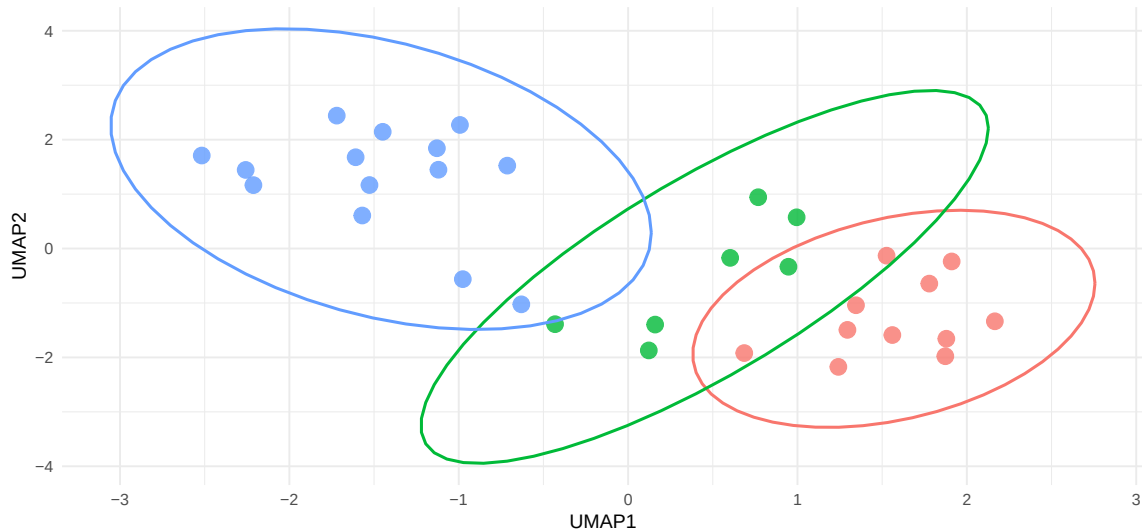
umap_df <- tibble(
  UMAP1 = umap_out$layout[, 1],
  UMAP2 = umap_out$layout[, 2],
  cyl   = factor(mtcars$cyl)
)

plot <- ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = cyl)) +
  geom_point(size = 4, alpha = 0.8) +
  stat_ellipse(type = "norm", linewidth = 0.8) +
  labs(title = "UMAP Visualization of mtcars") +
  theme_minimal()
```

UMAP, cont.

```
plot + theme(legend.position = "none")
```

UMAP Visualization of mtcars



PCA vs. t-SNE vs. UMAP

Method	Strengths	Weaknesses
PCA	Deterministic, interpretable loadings, fast	Linear only
t-SNE	Excellent local structure, good for clusters	Slow, non-deterministic, loses global structure
UMAP	Fast, preserves global + local structure	Non-deterministic, newer (fewer tools)

Section 8

Machine Learning: Classification

What is Classification?

Predicting a **categorical** outcome (e.g., disease vs. healthy, tumor subtype) from input features.

Key concepts:

- **Features (X)**: predictor variables (gene expression, clinical variables)
- **Labels (y)**: the class to predict
- **Training set**: data used to fit the model
- **Test set**: held-out data used to evaluate performance

Random Forest

Ensemble method that builds many decision trees and averages their predictions. **Widely used in bioinformatics** for:

- Gene selection / biomarker discovery
- Sample classification
- Feature importance ranking

```
library(randomForest)
set.seed(42)
# Predict transmission type (am) using other variables
rf_model <- randomForest(factor(am) ~ ., data = mtcars,
                          importance = TRUE, ntree = 500)
print(rf_model)
```

Call:

```
randomForest(formula = factor(am) ~ ., data = mtcars, importance = TRUE, ntree = 500)
      Type of random forest: classification
      Number of trees: 500
No. of variables tried at each split: 3
```



Random Forest: Feature Importance

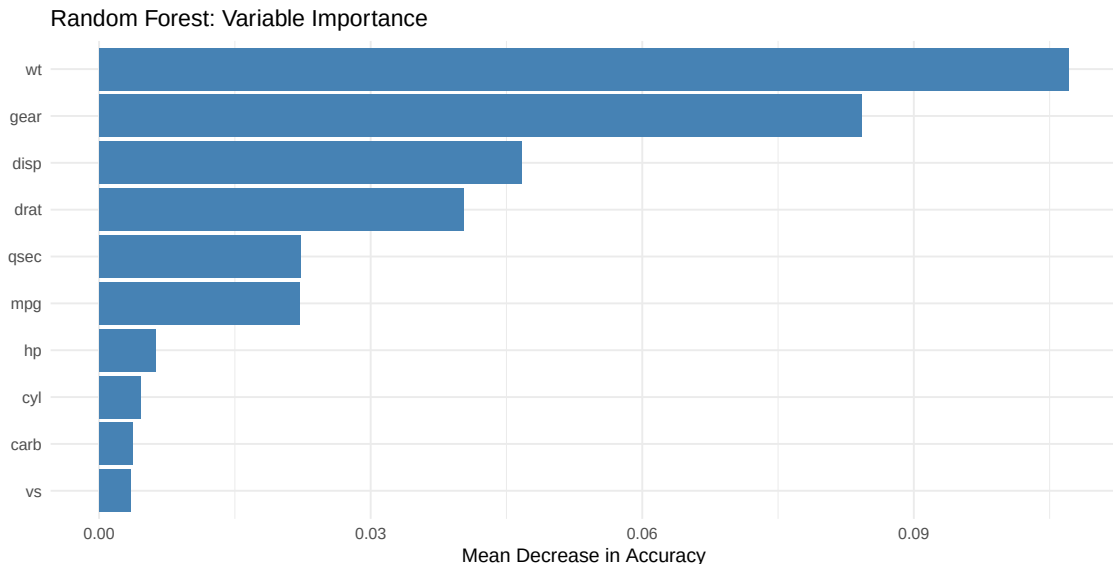
Which variables are most predictive?

```
# Extract and plot importance
imp_df <- as_tibble(rf_model$importance, rownames = "Variable") %>%
  arrange(desc(MeanDecreaseAccuracy))

plot <-
  ggplot(imp_df, aes(x = reorder(Variable, MeanDecreaseAccuracy),
                    y = MeanDecreaseAccuracy)) +
    geom_col(fill = "steelblue") +
    coord_flip() +
    labs(title = "Random Forest: Variable Importance",
         x = "", y = "Mean Decrease in Accuracy") +
    theme_minimal()
```

Random Forest: Feature Importance, cont.

```
plot
```



Random Forest: Confusion Matrix

How well did the model classify samples?

```
# Out-of-bag (OOB) confusion matrix  
rf_model$confusion
```

```
      0   1 class.error  
0 17   2   0.1052632  
1   2 11   0.1538462
```

Section 9

Cross-Validation & Model Evaluation

Why Cross-Validation?

- Prevents **overfitting** (model memorizes training data but fails on new data)
- Provides realistic estimate of model performance
- Essential for comparing models and tuning parameters

k-Fold Cross-Validation: Illustration

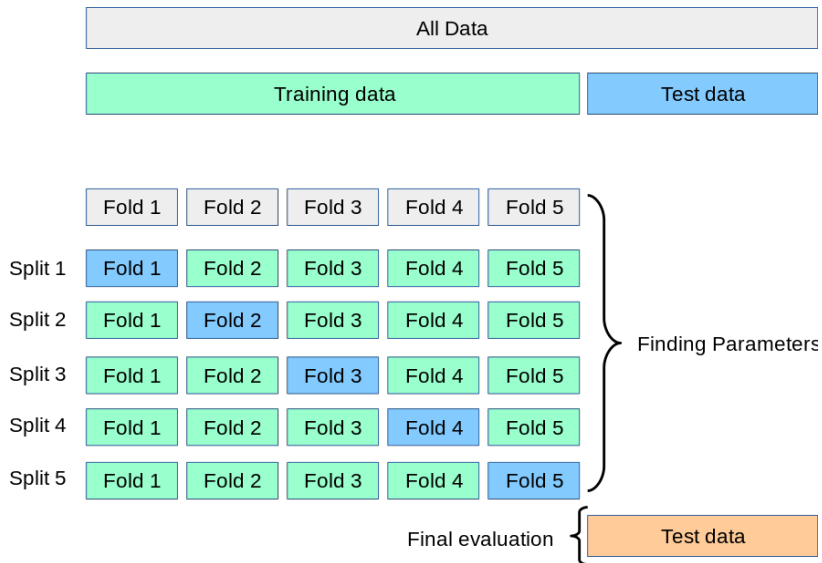


Figure 1: k-Fold Cross-Validation. Source: scikit-learn documentation

What is k-Fold CV?

Split data into **k** equal-sized folds:

- ① Train on $k-1$ folds, test on the remaining fold
- ② Repeat k times — each fold serves as test set **exactly once**
- ③ Final performance = **average** across all k iterations

Common choices: $k = 5$ or $k = 10$. Larger $k \rightarrow$ lower bias but higher variance.

k-Fold CV in R with caret

```
library(caret)

set.seed(42)

# 5-fold cross-validation
train_control <- trainControl(method = "cv", number = 5)

# Train random forest with CV
rf_cv <- train(factor(am) ~ ., data = mtcars,
               method = "rf",
               trControl = train_control,
               tuneLength = 3)
```

k-Fold CV Results

```
print(rf_cv)
```

Random Forest

32 samples

10 predictors

2 classes: '0', '1'

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 25, 26, 25, 27, 25

Resampling results across tuning parameters:

mtry	Accuracy	Kappa
2	0.9142857	0.8167273
6	0.8809524	0.7500606
10	0.8809524	0.7500606

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was mtry = 2.

Repeated k-Fold Cross-Validation

To reduce variability, repeat the entire k-fold process **N times** with different random splits:

- **k-fold CV**: k training runs, each sample tested once
- **N-times repeated k-fold CV**: $N \times k$ training runs, each sample tested N times
- More robust estimate, but computationally expensive

```
# 5-fold CV repeated 3 times
train_control_rep <- trainControl(
  method = "repeatedcv",
  number = 5,          # k = 5 folds
  repeats = 3          # repeat 3 times
)

rf_cv_rep <- train(factor(am) ~ ., data = mtcars,
  method = "rf",
  trControl = train_control_rep,
  tuneLength = 3)
```



Repeated k-Fold CV Results

```
print(rf_cv_rep)
```

Random Forest

32 samples

10 predictors

2 classes: '0', '1'

No pre-processing

Resampling: Cross-Validated (5 fold, repeated 3 times)

Summary of sample sizes: 25, 25, 25, 26, 27, 25, ...

Resampling results across tuning parameters:

mtry	Accuracy	Kappa
2	0.8580952	0.7079141
6	0.8714286	0.7382171
10	0.8580952	0.7079141

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was mtry = 6.

k-Fold vs. Repeated k-Fold CV

Feature	k-Fold CV	Repeated k-Fold CV
Training runs	k	$N \times k$
Each sample tested	1 time	N times
Variance of estimate	Moderate	Lower
Computational cost	Low	$N \times$ higher
Use when	Quick evaluation, large datasets	Final model evaluation, small datasets
caret method	method = "cv"	method = "repeatedcv"



Section 10

Classification: Types & Evaluation

Three Types of Prediction Problems

In machine learning, predictions fall into three categories:

Type	Target Variable	Examples
Binary classification	2 classes	Disease vs. Healthy, Responder vs. Non-responder
Multi-class classification	3+ classes (unordered)	Tumor subtype (Luminal A/B, Basal, HER2), Tissue type
Regression	Continuous value	Gene expression level, Drug IC50, Survival time

Binary Classification Metrics

Key evaluation metrics for two-class problems:

- **Accuracy:** $(TP + TN) / \text{Total}$ — overall proportion correct
- **Sensitivity (Recall):** $TP / (TP + FN)$ — ability to find all positives
- **Specificity:** $TN / (TN + FP)$ — ability to find all negatives
- **Precision:** $TP / (TP + FP)$ — proportion of predicted positives that are true positives
- **F1-score:** Harmonic mean of Precision and Recall — balanced metric
- **AUC-ROC:** Area under the ROC curve — overall discriminative ability

```
# Confusion matrix from random forest
pred <- predict(rf_model, mtcars)
actual <- factor(mtcars$am)

(cm <- confusionMatrix(pred, actual))
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	19	0
1	0	12



ROC Curve and AUC

ROC = Receiver Operating Characteristic. Plots True Positive Rate vs. False Positive Rate at all thresholds.

AUC = Area Under Curve (0.5 = random, 1.0 = perfect).

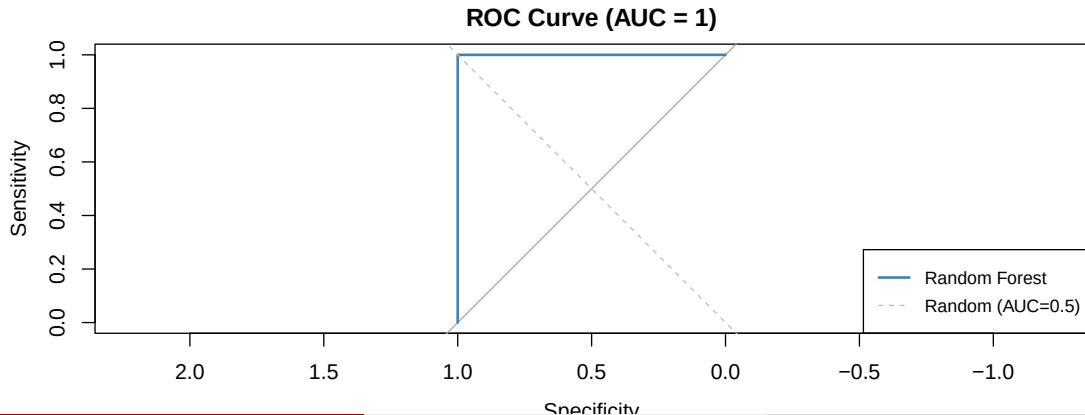
```
library(pROC)

# Get predicted probabilities
pred_prob <- predict(rf_model, mtcars, type = "prob")[, 2]

# Compute ROC and AUC
roc_obj <- roc(actual, pred_prob)
auc_val <- auc(roc_obj)
```

ROC Curve

```
# Plot ROC curve
plot(roc_obj, main = paste0("ROC Curve (AUC = ", round(auc_val, 3), ")"),
     col = "steelblue", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray") # random classifier
legend("bottomright", legend = c("Random Forest", "Random (AUC=0.5)"),
     col = c("steelblue", "gray"), lty = c(1, 2), lwd = c(2, 1), cex = 0.8)
```



Multi-class Classification Metrics

For problems with **3+ classes**, we extend binary metrics:

- **Overall Accuracy:** total correct / total samples
- **Per-class metrics:** Sensitivity, Precision, F1 for each class
- **Macro-average:** average of per-class metrics (each class equally weighted)
- **Weighted-average:** weighted by class size (accounts for class imbalance)
- **Confusion matrix:** $N \times N$ matrix showing per-class predictions

```
# Multi-class confusion matrix from caret
# Example: predict cyl (4/6/8) using all features
set.seed(42)
rf_multi <- randomForest(factor(cyl) ~ ., data = mtcars, importance = TRUE)
pred_multi <- predict(rf_multi, mtcars)
cm_multi <- confusionMatrix(pred_multi, factor(mtcars$cyl))
```



Multi-class Confusion Matrix

```
cm_multi$byClass # per-class metrics
```

	Sensitivity	Specificity	Pos Pred Value	Neg Pred Value	Precision	Recall
Class: 4	1	1	1	1	1	1
Class: 6	1	1	1	1	1	1
Class: 8	1	1	1	1	1	1

	F1	Prevalence	Detection Rate	Detection	Prevalence	Balanced Accuracy
Class: 4	1	0.34375	0.34375	0.34375		1
Class: 6	1	0.21875	0.21875	0.21875		1
Class: 8	1	0.43750	0.43750	0.43750		1

Regression Metrics

For predicting **continuous** values:

- **RMSE** (Root Mean Squared Error): $\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$ — same unit as target, penalizes large errors
- **MAE** (Mean Absolute Error): $\frac{1}{n} \sum |y_i - \hat{y}_i|$ — less sensitive to outliers
- R^2 (Coefficient of Determination): proportion of variance explained (0 to 1, higher is better)
- **Spearman** ρ : rank correlation between predicted and actual

Regression Metrics, cont.

```
# Regression example: predict mpg
set.seed(42)
rf_reg <- randomForest(mpg ~ ., data = mtcars, importance = TRUE)
pred_reg <- predict(rf_reg, mtcars)

# Calculate metrics
rmse <- sqrt(mean((mtcars$mpg - pred_reg)^2))
mae <- mean(abs(mtcars$mpg - pred_reg))
r2 <- cor(mtcars$mpg, pred_reg)^2

cat(sprintf("RMSE = %.3f\nMAE = %.3f\nR2 = %.3f\n", rmse, mae, r2))
```

```
RMSE = 1.197
MAE = 0.947
R2 = 0.964
```

Summary: Choosing the Right Metric

Prediction Type	Primary Metrics	R Package
Binary classification	Accuracy, Sensitivity, Specificity, F1, AUC	caret, pROC
Multi-class classification	Overall accuracy, Macro/Weighted F1, Confusion matrix	caret
Regression	RMSE, MAE, R^2	caret, base R

Section 11

Traditional Machine Learning Algorithms

Common Traditional ML Algorithms

Traditional ML relies on **hand-crafted features** and statistical principles. Here are the key categories:

Category	Algorithms	Key Idea	Bioinformatics Use
Linear Models	Linear/Logistic Regression, Lasso, Ridge, Elastic Net	Fit linear relationships; regularization for feature selection	Gene-disease association, biomarker selection
Tree-based	Decision Tree, Random Forest, Gradient Boosting (XGBoost, LightGBM)	Ensemble of decision trees	Disease classification, feature importance ranking
Support Vector	SVM (linear, RBF kernel)	Find optimal separating hyperplane	Cancer subtype classification, protein classification
Nearest Neighbor	k-NN	Classify by majority vote of k nearest neighbors	Sample similarity, patient stratification
Naive Bayes	Gaussian/Multinomial NB	Probabilistic classifier assuming feature independence	Text mining (literature), sequence motif classification



When to Use Traditional ML

- **Limited data** (hundreds to thousands of samples)
- **Interpretability** is important (which genes/features drive prediction?)
- **Structured/tabular data** (gene expression matrix, clinical variables)
- **Well-defined features** exist

Section 12

Deep Learning: CNN-based Modern ML

From Traditional ML to Deep Learning

Traditional ML requires **manual feature engineering** — domain experts must decide which features matter. Deep learning **learns features automatically** from raw data through multiple layers of abstraction.

Convolutional Neural Networks (CNNs)

CNNs are the backbone of modern biomedical image analysis:

- **Convolutional layers:** learn spatial patterns (edges → shapes → objects)
- **Pooling layers:** reduce dimensionality, provide translation invariance
- **Fully connected layers:** make final predictions

Key CNN Architectures & Biomedical Applications

Architecture	Year	Key Innovation	Biomedical Applications
LeNet	1998	First CNN — digit recognition	Histopathology image classification
AlexNet	2012	Deep CNN + ReLU + Dropout	Medical image diagnosis (X-ray, CT)
VGGNet	2014	Very deep (16-19 layers), small filters	Tissue classification, cell detection
ResNet	2015	Skip connections (solves vanishing gradient)	Radiology (MRI/CT analysis), pathology
U-Net	2015	Encoder-decoder with skip connections	Image segmentation (tumor boundary, cell nucleus)
Inception (GoogLeNet)	2014	Multi-scale parallel convolutions	Retinal disease detection, dermatology
DenseNet	2017	Dense connections between all layers	Chest X-ray analysis (COVID-19, TB)
EfficientNet	2019	Compound scaling (depth, width, resolution)	General medical image analysis
Vision Transformer (ViT)	2020	Transformer architecture for images	Histopathology whole-slide image analysis



Beyond CNNs: Other Modern DL Architectures

Architecture	Best For	Biomedical Applications
RNN / LSTM / GRU	Sequential data	DNA/protein sequence analysis, ECG/EEG
Transformers	Long-range dependencies	Protein structure (AlphaFold), DNA language models (DNABERT)
Graph Neural Networks (GNN)	Graph-structured data	Drug-target interaction, protein-protein interaction, molecular property prediction
Autoencoders	Dimensionality reduction, denoising	scRNA-seq denoising, batch correction
Generative Models (GAN, VAE)	Data generation	Drug design (de novo molecules), medical image synthesis



Traditional ML vs. Deep Learning

Aspect	Traditional ML	Deep Learning (CNN/DL)
Feature engineering	Manual — domain expert required	Automatic — learned from raw data
Data requirements	100s – 10,000s samples	10,000s – millions of samples
Interpretability	High (feature importance, coefficients)	Low (“black box”; improving with SHAP, Grad-CAM)
Computational cost	Low (CPU sufficient)	High (GPU often required)
Performance on images/text	Moderate (with manual features)	State-of-the-art
Performance on tabular data	Often competitive or better	Can overfit on small tables
Training time	Seconds to minutes	Hours to days
Overfitting risk	Lower (simpler models)	Higher (requires regularization, augmentation)
R ecosystem	Mature (caret, tidymodels, mlr3)	Growing (keras, torch, tensorflow)



Choosing the Right Approach



Tip

Rule of thumb: Start with traditional ML. Move to deep learning when:

- 1 You have **large datasets** (10,000+ samples)
- 2 Working with **images, sequences, or unstructured data**
- 3 Traditional ML performance **plateaus** despite feature engineering
- 4 You need **end-to-end learning** from raw data to prediction

Section 13

Summary

What We Covered

Topic	Key Functions / Packages
Descriptive stats	<code>summary()</code> , <code>summarise()</code> , <code>group_by()</code> , <code>tableone</code>
Hypothesis testing	<code>t.test()</code> , <code>wilcox.test()</code> , <code>aov()</code> , <code>TukeyHSD()</code> , <code>kruskal.test()</code> , <code>chisq.test()</code>
Multiple testing	<code>p.adjust()</code> with “bonferroni” and “BH”
Correlation	<code>cor.test()</code> (Pearson, Spearman), <code>corrplot</code>
Clustering	<code>kmeans()</code> , <code>hclust()</code> , <code>heatmap()</code> , elbow method
Dimensionality reduction	<code>prcomp()</code> (PCA), <code>Rtsne</code> (t-SNE), <code>umap</code> (UMAP)
Classification	<code>randomForest</code> , <code>caret::train()</code>
Model evaluation	<code>confusionMatrix()</code> , cross-validation



Key Takeaways

- ① **Always start with descriptive statistics** — understand your data before testing
- ② **Choose the right test** for your data type and distribution
- ③ **Correct for multiple testing** in high-dimensional bioinformatics data
- ④ **PCA/t-SNE/UMAP** are essential for exploring high-dimensional data
- ⑤ **Random Forest** is a robust, interpretable classifier widely used in bioinformatics
- ⑥ **Cross-validate** to get honest performance estimates

Next Talk

Talk 10: Regression & Survival Analysis

Homework

Complete Homework_AI_powered_R/talk09-homework.qmd